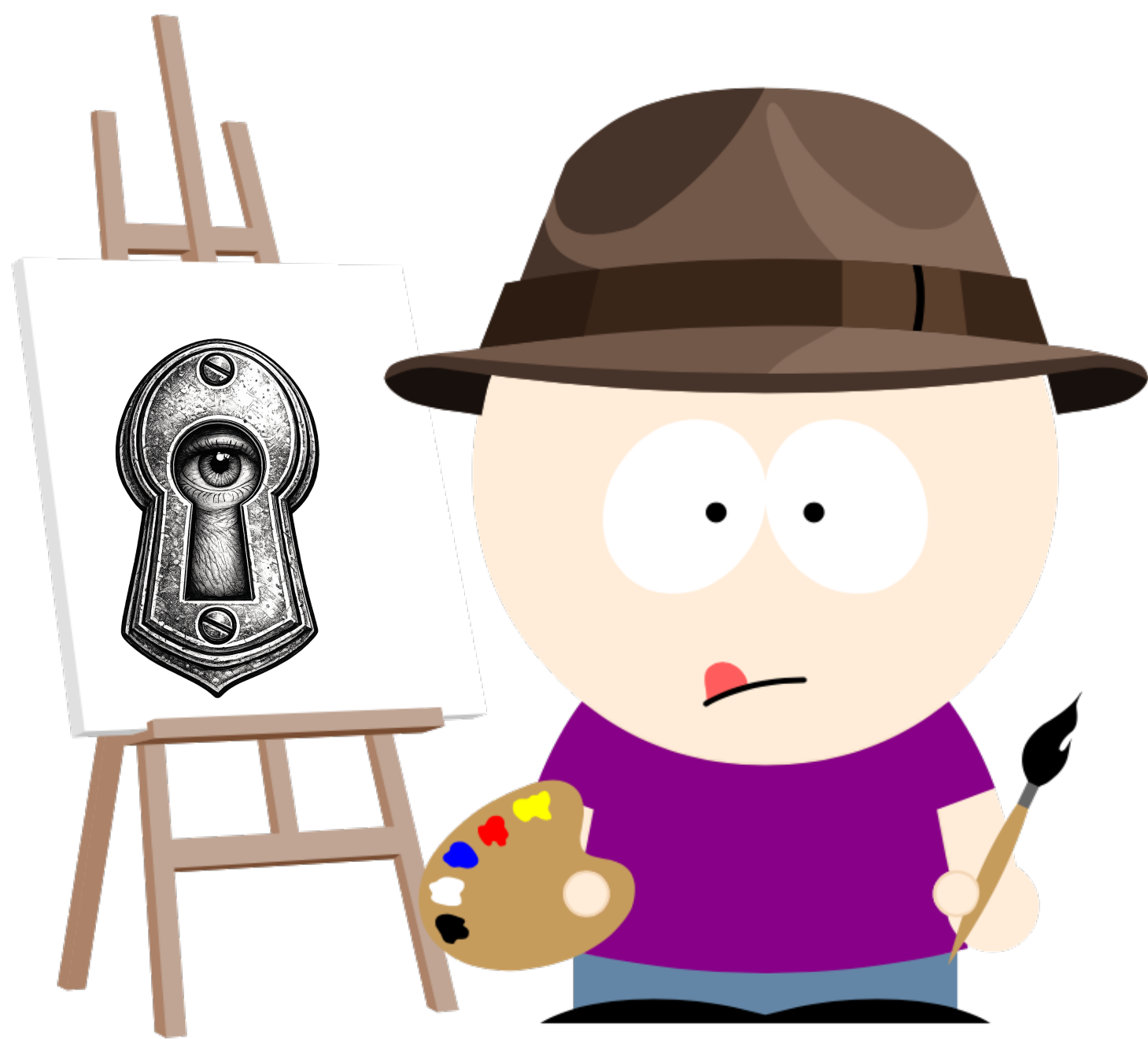




IN-CLASS EXERCISES



CODE OPTIMIZATION 1

Download exercises

1. Go to

<https://ligerlabs.org/compilers.html>

2. Download the file

`opt-1-exercises.zip`

3. Open up a terminal and Unzip the file

```
> unzip opt-1-exercises.zip
```

```
> cd opt-1-exercises
```

```
> ls
```

Task 1

- Define these terms:
 1. Local optimization
 2. Global optimization
 3. Inter-procedural optimization
 4. Peephole optimization
 5. Semantics-preserving transformation
 6. Profile -guided code optimization
 7. Runtime optimization

Task 2

- Explain what each of these local code transformations do:
 1. Local common subexpression elimination.
 2. Local copy propagation.
 3. Local dead-code elimination.
 4. Algebraic optimization.
 5. Jump-to-jump removal.
 6. Reduction in strength.

Task 3 (a)

- Have a look at the program `peephole.c`.
- This program implements a simple Quad IR.
- To the left is a program "PRINT 42+67+11" and to the right the corresponding C array of Quads:

```
[0]: R0 = LOADINT 42
[1]: R1 = LOADINT 67
[2]: R2 = R0 ADD R1
[3]: R3 = LOADINT 11
[4]: R4 = R2 ADD R3
[5]: PRINT R4
[6]: EXIT
```

```
struct Quad code3[] = {
    {0, LOADINT, 42, 0},
    {1, LOADINT, 67, 0},
    {2, ADD, 0, 1},
    {3, LOADINT, 11, 0},
    {4, ADD, 2, 3},
    {0, PRINT, 4, 0},
    {0, EXIT, 0, 0}
};
```

Task 3 (b)

1. Write a routine `constFold()` that implements a constant folding peephole optimizer over the Quad array.
2. Write a routine `removeJumps2Jumps()` that simplifies jumps 2 jumps over the Quad array.
3. Write a routine `algebraicSimplification()` that implements a peephole optimizer that uses a few algebraic identities.

Task 4

- Consider the program on the next page and in `task4.s`.
- Simulate the execution of a peephole optimizer by repeatedly sweeping a window of size 3 over the code, and simplifying the program as much as possible.
- Consider these transformations:
 - Algebraic simplification
 - Reduction in strength
 - Constant folding
 - Copy propagation

la	\$2,	X	
la	\$3,	X	
lw	\$4,	0 (\$2)	
addiu	\$4,	\$4,	1
mulo	\$4,	\$4,	2
sll	\$4,	\$4,	1
sw	\$4,	0 (\$2)	
lw	\$4,	X	
neg	\$4,	\$4	
neg	\$4,	\$4	
move	\$5,	\$4	
sw	\$5,	X	
lw	\$4,	X	
lw	\$5,	X	
sub	\$4,	\$4,	\$5
sw	\$4,	X	

Task 5

- Optimize these operations by turning them into combinations of shifts and adds:

1. $X * 128$

2. $X / 32$

3. $X * 120$

Task 6

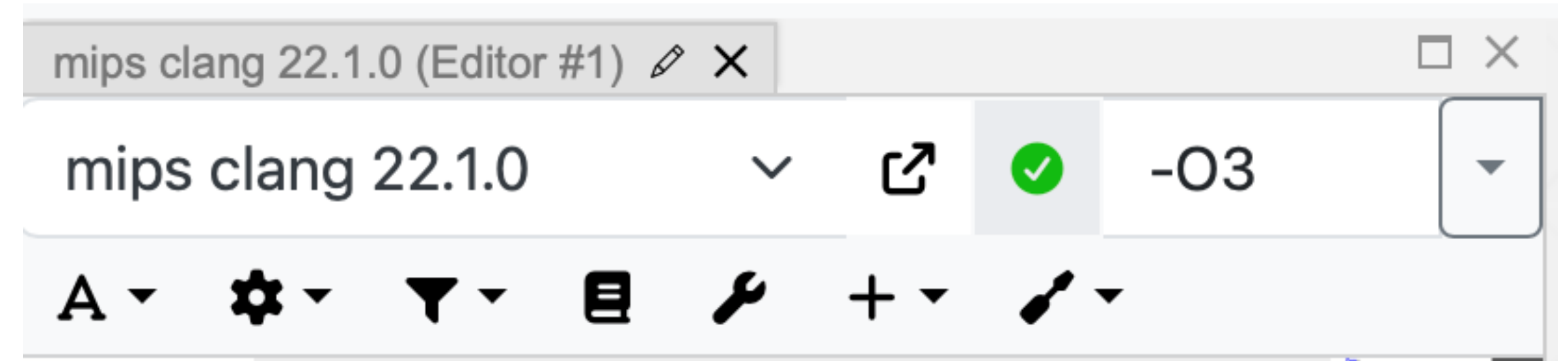
- How do compilers implement integer division by a constant? Let's find out!

1. Go to `godbolt.org`.

2. Enter this program:

```
int foo (int x) {  
    return x / 11;  
}
```

3. Set optimization level to `-O3`:



4. Examine the code generated from `x/11`.

Can you figure out what it does?

5. Experiment with other constants.



DEFINITIONS AND ALGORITHMS

Algebraic Simplification

Identity Property

$$x + 0 \Rightarrow x$$

$$x - 0 \Rightarrow x$$

$$x * 1 \Rightarrow x$$

$$x / 1 \Rightarrow x$$

Zero Property

$$x - x \Rightarrow 0$$

$$x * 0 \Rightarrow 0$$

$$0 / x \Rightarrow 0$$

Double Negation

$$-(-x) \Rightarrow x$$

Distributive Law

$$a*x + b*x \Rightarrow (a+b)*x$$

Reduction in Strength

$$x ** 2 \Rightarrow x * x$$

$$x * 2 \Rightarrow x + x$$

$$f / 2.0 \Rightarrow f * 0.5$$

Peephole Optimization Algorithm

- Usually done at the machine code level.
- Could also be done at the intermediate code level.

1. Examine a “window” (of size n) of instructions.

2. Improve the code in the window.

3. Slide the window down one instruction.

4. Repeat until from 1 until done.

lw	\$2, 4(\$fp)
lw	\$3, 8(\$fp)
addiu	\$4, \$2, 16
addiu	\$4, \$4, -16
addu	\$4, \$4, \$3
sw	\$4, 12(\$fp)

Peephole Optimization

Can we
optimize
this?

```
lw      $2, 4($fp)
```

```
lw      $3, 8($fp)
```

```
addiu   $4, $2, 16
```

```
addiu   $4, $4, -16
```

```
addu    $4, $4, $3
```

```
sw      $4, 12($fp)
```

COLLBERG.CS.ARIZONA.EDU

LIGERLABS.ORG

SUPPORTED BY
NSF SATC/TTP-1525820
SATC/EDU-2029632

COPYRIGHT © 2024-2026

CHRISTIAN COLLBERG

UNIVERSITY OF ARIZONA